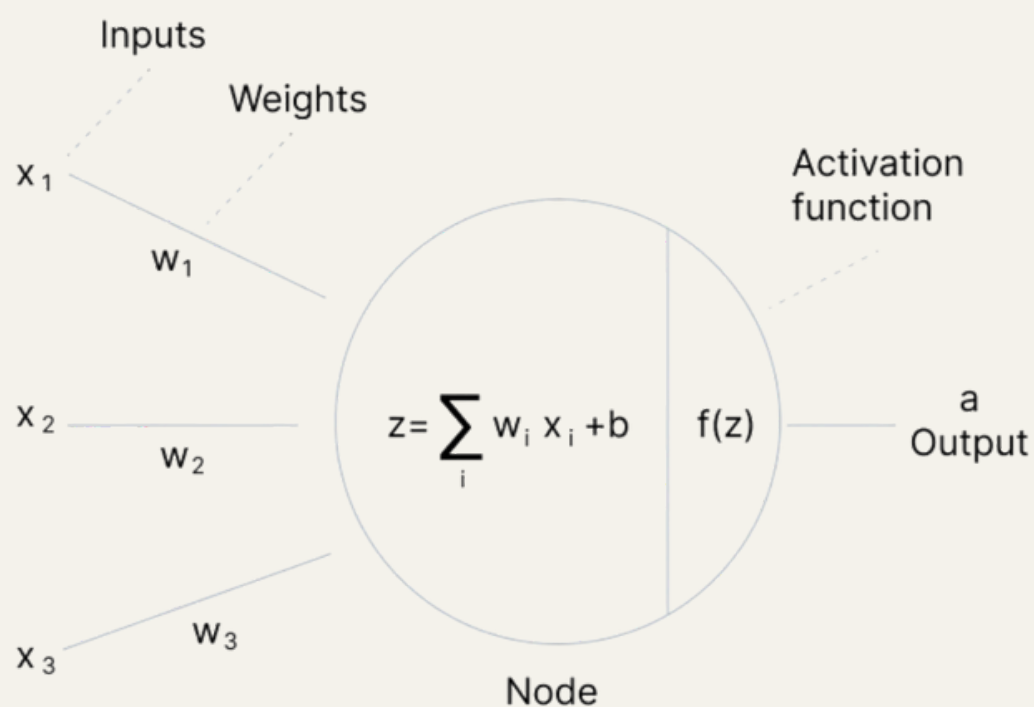


Activation Functions: Breathing Life into Neural Networks

WHAT IS ACTIVATION FUNCTION?

- It's mathematical function which applied to **weighted sum of the inputs of the neurons**.
- In simpler terms, an activation function is a mathematical gate that determines whether a neuron should be activated or not. It introduces non-linearity into neural networks, enabling them to learn complex patterns and solve real-world problems.
- Without activation functions, even the deepest neural network would behave like a simple linear regression model—unable to capture the intricate relationships in data.



WHY ACTIVATION FUNCTION NEEDED?

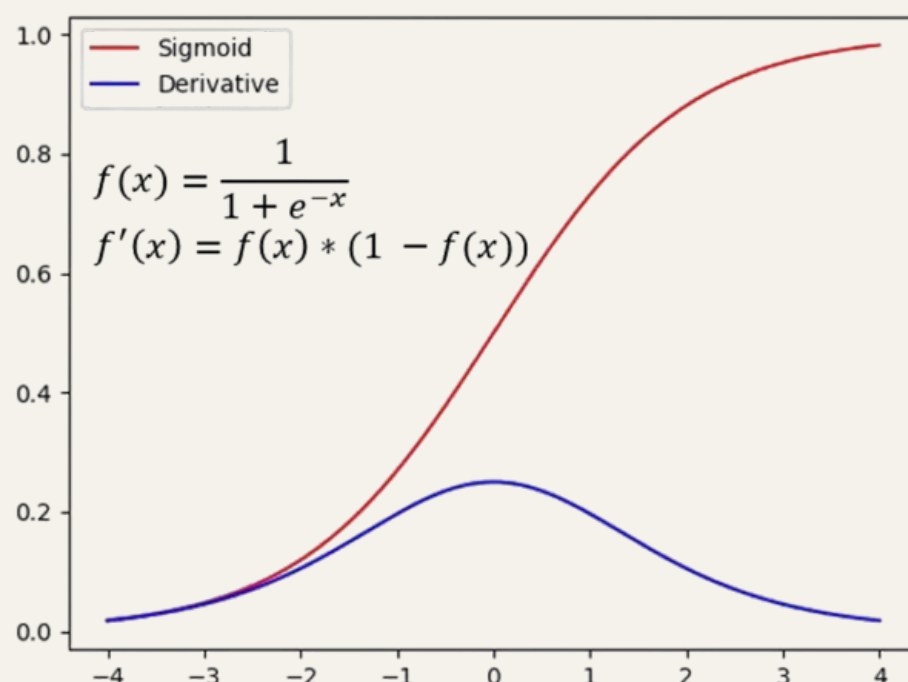
- **Introduce Non-Linearity:** Real-world data is rarely linear. Activation functions allow neural networks to approximate complex, non-linear relationships between inputs and outputs.
- **Enable Deep Learning:** Without non-linear activations, stacking multiple layers would be pointless—the network would collapse into a single linear transformation, defeating the purpose of depth.
- **Gradient Flow Management:** Different activation functions handle gradients differently, directly impacting how effectively the network learns during backpropagation.



TYPES OF ACTIVATION FUNCTION

Sigmoid: The Classic Pioneer

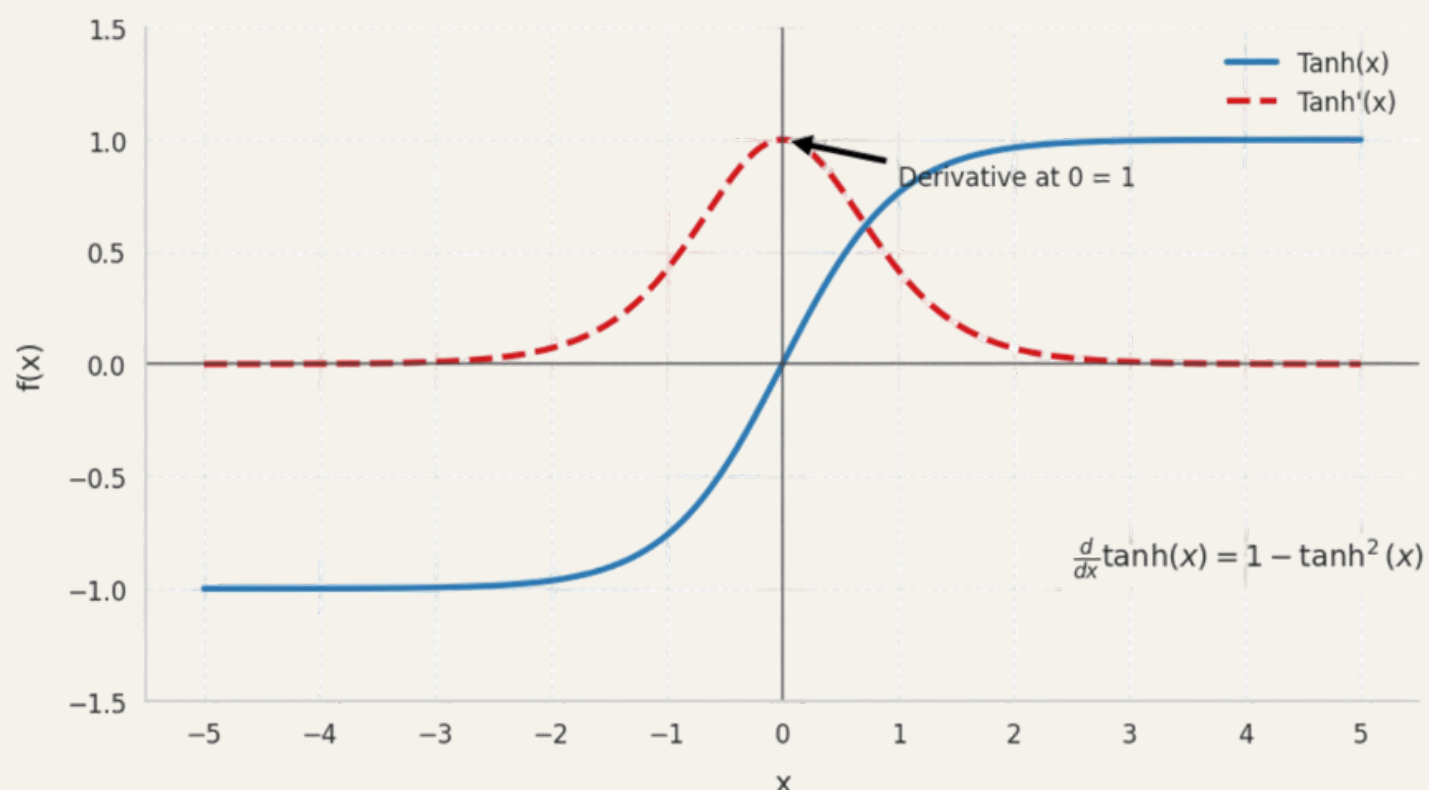
- **Formula:** $\sigma(x) = 1 / (1 + e^{(-x)})$
- **Range:** (0, 1)
- **Characteristics:**
 - **Smooth gradient**, easy to interpret as probabilities
 - Perfect for binary classification output layers
 - **Suffers from vanishing gradient problem** in deep networks
 - Outputs **not zero-centered**, causing slower convergence
 - Computationally expensive (exponential calculation)
- **Best Use Case:** Output layer for binary classification where you need probability scores between 0 and 1.



TYPES OF ACTIVATION FUNCTION

Tanh: The Improved Sigmoid

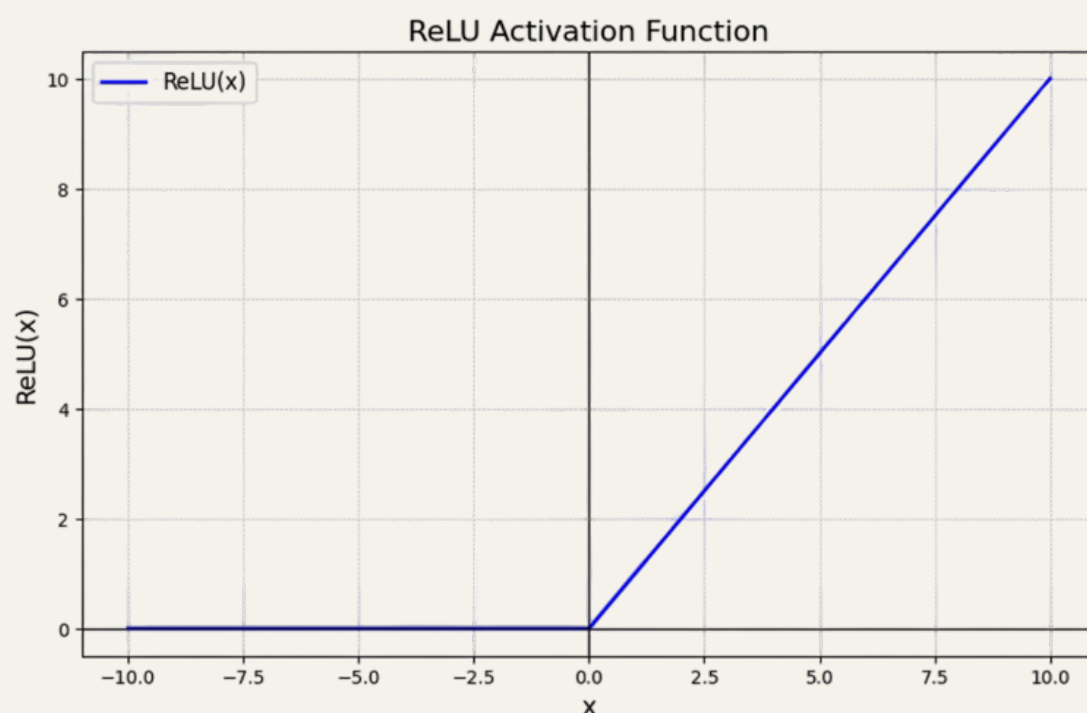
- **Formula:** $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$
- **Range:** $(-1, 1)$
- **Characteristics:**
 - **Zero-centered output** (better than Sigmoid)
 - Stronger gradients than Sigmoid
 - Works well in hidden layers of shallow networks
 - Still **suffers from vanishing gradient problem**
 - Computationally expensive
- **Key Improvement Over Sigmoid:** The zero-centered nature means data isn't always pushed toward positive values, leading to more balanced and faster learning.



TYPES OF ACTIVATION FUNCTION

ReLU: The Game Changer

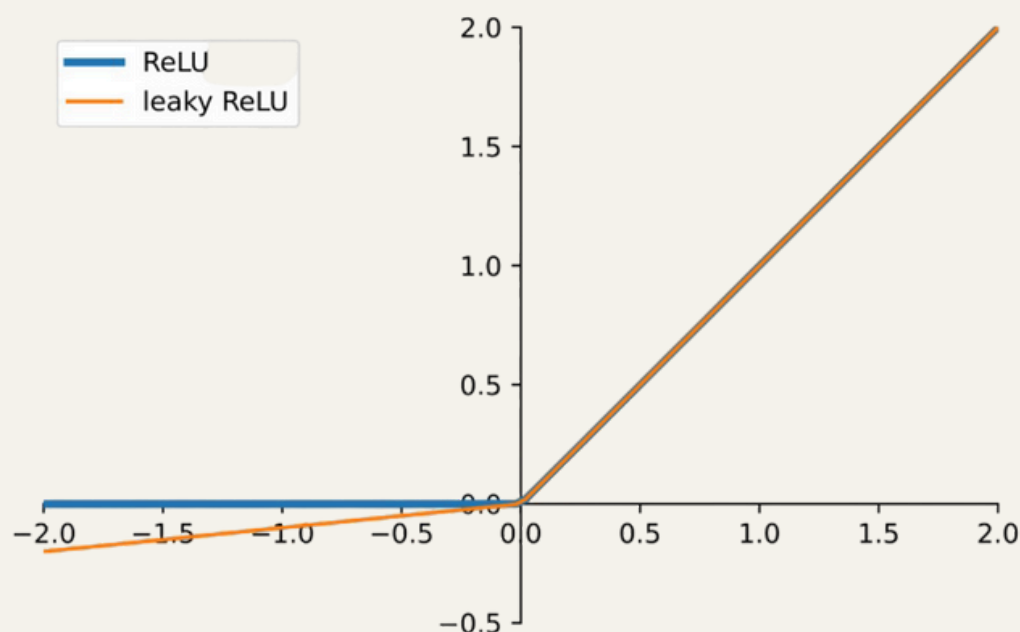
- **Formula:** $\text{ReLU}(x) = \max(0, x)$
- **Range:** $[0, \infty)$
- **Characteristics:**
 - Extremely simple and fast to compute
 - **Solves vanishing gradient problem** (for positive values)
 - Promotes sparsity (neurons with negative input become 0)
 - Converges much faster than Sigmoid/Tanh
 - **Dying ReLU problem:** Neurons can get stuck at 0 and never activate again
 - Not zero-centered
- **Why It Became the Standard:** Its computational simplicity and effectiveness in solving gradient problems made it the go-to activation for deep networks, accelerating the deep learning revolution.



TYPES OF ACTIVATION FUNCTION

Leaky ReLU: Fixing the "Dying" Problem

- **Formula:** $\text{Leaky ReLU}(x) = \max(\alpha x, x)$, where $\alpha \approx 0.01$
- **Range:** $(-\infty, \infty)$
- **Characteristics:**
 - **Solves dying ReLU problem** by allowing small negative gradients
 - Maintains computational efficiency
 - All **neurons remain active** during training
 - Performance depends on choosing the right α value
 - Slightly more complex than standard ReLU
- **Key Innovation:** By giving a small slope (α) to negative values, neurons that receive negative inputs can still learn and contribute, preventing permanent "death."



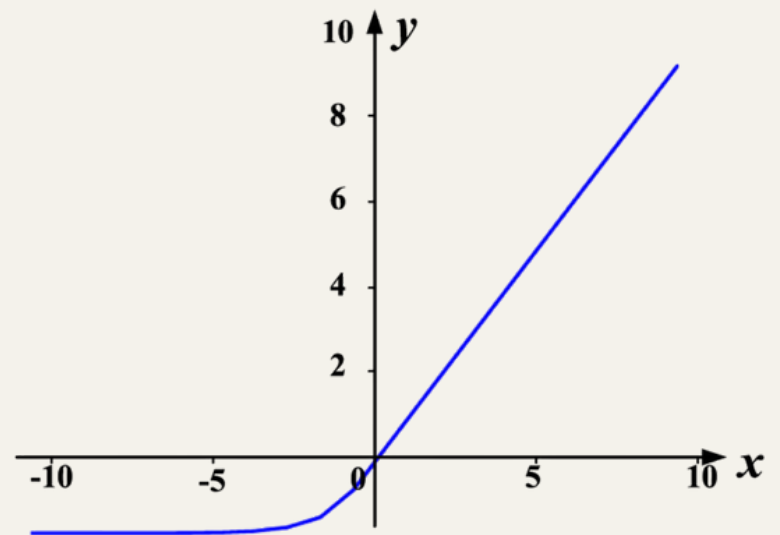
TYPES OF ACTIVATION FUNCTION

ELU: The Smooth Operator

- **Formula:**

- $\text{ELU}(x) = x$, if $x > 0$
- $\text{ELU}(x) = \alpha(e^x - 1)$, if $x \leq 0$

- **Range:** $(-\alpha, \infty)$



- **Characteristics:**

- Smooth everywhere, including at zero
- Produces negative outputs, helping with zero-centered mean
- Reduces bias shift and speeds up learning
- More robust to noise than ReLU
- Computationally more expensive
- May slow down training compared to ReLU

- **What Makes It Special:** ELU combines the benefits of ReLU (fast positive part) with smooth negative saturation, pushing mean activations closer to zero and accelerating convergence.

Decision Guide

How to Choose the Right Activation Function?

🎯 For Hidden Layers:

- Start with ReLU - It's fast, effective, and works 90% of the time
- Switch to Leaky ReLU if you notice many dead neurons
- Try ELU if you need smoother learning and can afford the computational cost

🎯 For Output Layers:

- Binary Classification → Sigmoid (outputs probability)
- Multi-class Classification → Softmax
- Regression → Linear (no activation) or ReLU for positive outputs

🎯 General Rule of Thumb:

- Start with ReLU for hidden layers
- Monitor training - if learning stalls, try Leaky ReLU or ELU
- For output layers, let the task dictate (Sigmoid for binary, Softmax for multi-class)



Conclusion

Conclusion: Choose Wisely, Experiment Often

- In the world of neural networks, activation functions are the unsung heroes that make learning possible. From Sigmoid's pioneering days to ReLU's revolution and beyond, each function brings unique strengths to the table.
- **Key Takeaways:**
 -  ReLU is your starting point - simple, fast, effective
 -  Understand the problem - dying neurons? Try Leaky ReLU or ELU
 -  Output layers matter - match activation to your task
 -  Don't be afraid to experiment - one function doesn't rule them all
- The neural network revolution was built on smart choices in activation functions. Make yours count. 

GOOD
TIMES

@AkashChandrasekar24

What do you think
about the future of
AI ?



SHARE
YOUR
THOUGHTS



SAVE
FOR
LATER



LIKE
THIS
POST